FIAP — Engenharia de Software

Dynamic Programming

Checkpoint 1 — Relatório Técnico

Simulação de Centro de Distribuição Logístico e Sistema de Fretes e Entregas

Nome	RM	RA
Lucas Zanella	563880	PAR
Daniel Oliveira	566284	PAR
João Pedro Marcilio	561603	ÍMPAR
Ben Hur Lung	561564	PAR
Felipe Campos	562752	PAR

São Paulo — 2026

Simulação Centro de Distribuição Logístico

1. Objetivo

Desenvolver uma solução em Python utilizando estruturas de dados (tuplas, listas, dicionários, deque e DataFrame), ordenação e conceitos práticos de eficiência (Big O), aplicados a problemas reais de logística no Brasil.

O sistema simula um centro de distribuição com fila de pedidos, priorizando entregas por urgência e organizando as saídas logísticas com base nos dados fornecidos em CSV.

2. Estruturas de Dados Utilizadas

2.1 Tuplas

Cada linha do CSV é convertida em uma tupla imutável representando um pedido completo:

(pedido_id, cidade_destino, produto, categoria, quantidade, valor_unitario, urgencia, tempo_estimado_horas, modal, status_pagamento)

Tuplas foram escolhidas por sua imutabilidade — os dados de um pedido não devem ser alterados após o carregamento.

2.2 Listas

A lista pedidos_tuplas armazena todas as tuplas de pedidos. Além dela, é utilizada uma lista de prioridades ["alta", "media", "baixa"], que permite o fatiamento para alimentar a recursão via prioridades[1:].

2.3 Dicionários

O dicionário é utilizado como estrutura intermediária na função recursiva. Foi criado um dicionário onde as chaves são as urgências ('alta', 'media', 'baixa') e os valores são estruturas deque agrupando os pedidos de cada categoria antes da unificação.

2.4 Deque

A deque (double-ended queue) é a estrutura central da fila de prioridade final. A partir da separação inicial, uma fila única é consolidada tendo a urgência "media" como base:

- appendleft(item) — insere os pedidos de urgência alta no início da fila.
- append(item) — insere os pedidos de urgência baixa no final da fila.

Ambas as operações de inserção possuem complexidade O(1). A escolha da deque justifica-se por garantir a ordem exata de atendimento da fila com eficiência O(1) ao retirar o primeiro elemento, algo que listas comuns não fariam sem realocar todos os itens (O(n)).

2.5 DataFrame (Pandas)

Utilizado para carregar o CSV, realizar agrupamentos e alimentar as visualizações gráficas de análise dos dados logísticos.

3. Recursão — Organização da Fila

A função organizar_fila é recursiva e segue a estrutura clássica de caso base e caso recursivo:

- Caso base: lista de prioridades vazia (if not prioridades:) — retorna o dicionário com os grupos montados.
- Caso recursivo: utiliza um laço for para encontrar todos os pedidos da categoria atual, insere na deque correspondente via append(), e chama a função para a próxima prioridade da lista.

Essa abordagem garante que todos os pedidos sejam processados de forma ordenada e sem uso de laços explícitos.

4. Análise de Complexidade (Big O)

Operação	Complexidade	Justificativa
Carregamento do CSV	O(n)	Percorre todas as n linhas uma vez
Conversão para tuplas	O(n)	Uma tupla criada por linha
Ordenação dos pedidos	O(n log n)	Algoritmo Merge Sort
Organização da fila (recursão)	O(n)	A lista inteira é percorrida para cada uma das 3 prioridades. O(3n) simplifica para O(n).
Inserção na deque	O(1)	appendleft e append são constantes
Geração dos gráficos	O(n)	Percorre o DataFrame uma vez por gráfico

5. Análise Gráfica

5.1 Quantidade de Pedidos por Urgência

Gráfico de barras mostrando a distribuição dos pedidos entre os níveis de urgência alta, média e baixa.

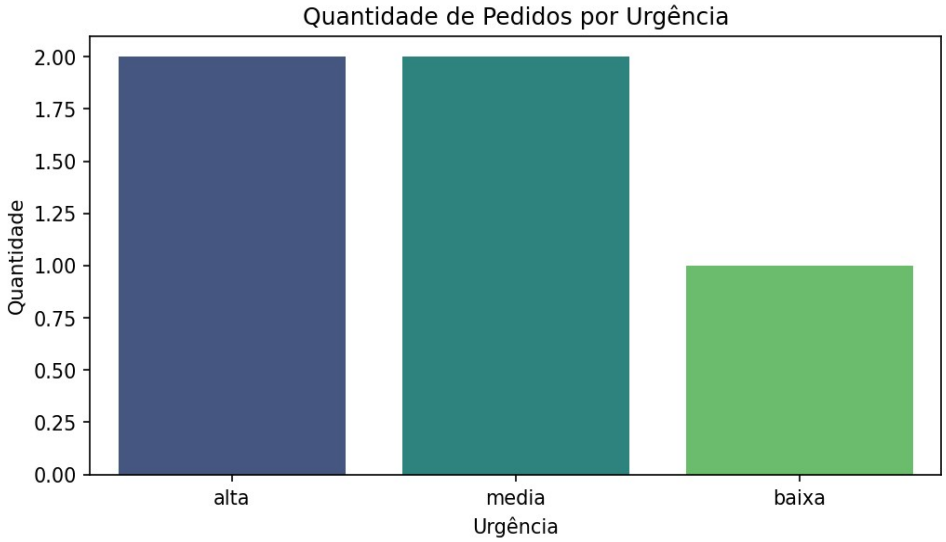

Figura 1 — Quantidade de Pedidos por Urgência

5.2 Valor Total por Cidade de Destino

Gráfico de barras com o valor unitário agregado por cidade de destino, permitindo identificar os destinos de maior valor logístico.

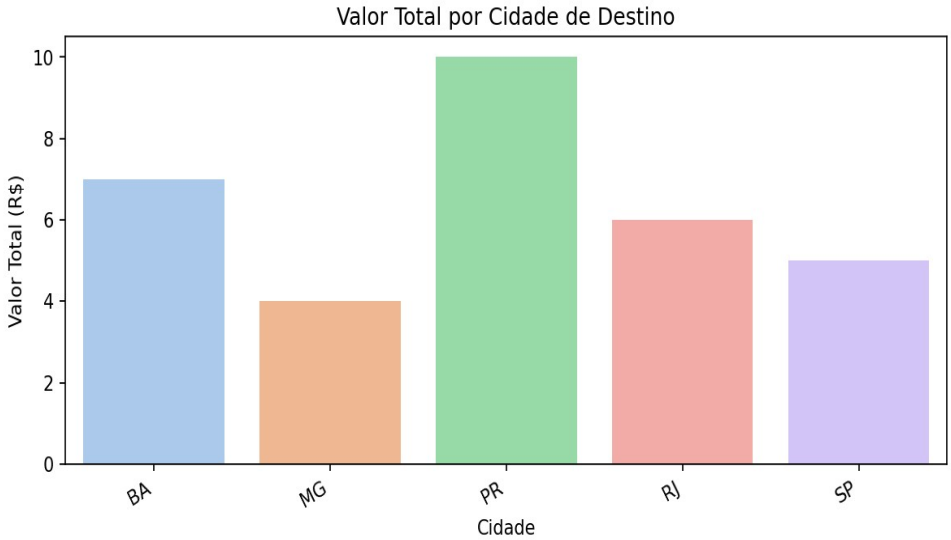

Figura 2 — Valor Total por Cidade de Destino

5.3 Distribuição de Quantidade por Destino

Gráfico de pizza representando a proporção de quantidade transportada para cada destino, evidenciando os estados com maior volume de entrega.

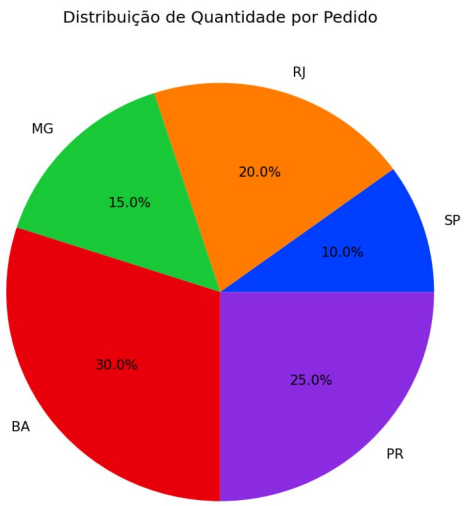

Figura 3 — Distribuição de Quantidade por Destino

6. Conclusão

A solução desenvolvida cumpre todos os requisitos do Checkpoint 1, aplicando de forma integrada as estruturas de dados exigidas: tuplas para imutabilidade dos pedidos, listas para armazenamento e fatiamento, dicionários no carregamento do CSV, deque para a fila de prioridade e DataFrame para análise e visualização.

A recursão implementada na função organizar_fila garante a priorização correta dos pedidos por urgência, com complexidade O(n) e inserções O(1) na deque. Os gráficos gerados com Seaborn e Matplotlib oferecem uma visão analítica clara dos dados logísticos trabalhados.

Simulação Sistema de Fretes e Entregas

1. Objetivo

Desenvolver uma solução em Python utilizando estruturas de dados (tuplas, listas, dicionários, deque e DataFrame), ordenação e conceitos práticos de eficiência (Big O), aplicados a problemas reais de logística no Brasil.

O sistema simula um sistema de gerenciamento de fretes e entregas, priorizando as cargas com base em uma hierarquia de critérios: nível de cliente (VIP ou Normal), prazos mais curtos e maiores valores de frete.

2. Estruturas de Dados Utilizadas

2.1 Tuplas

Cada linha do CSV é convertida em uma tupla imutável representando os dados completos de uma entrega:

(entrega_id, origem, destino, tipo_carga, peso_kg, valor_frete, prazo_dias, prioridade_cliente, situacao_rota, transportadora)

Tuplas foram escolhidas por sua imutabilidade, os dados de um pedido não devem ser alterados após o carregamento.

2.2 Listas

A lista cargas_tuplas armazena todas as tuplas originais de pedidos e serve como base para a ordenação. Além disso, listas são usadas no controle da recursão para armazenar e fatiar as categorias de prioridade (categorias[1:]).

2.3 Dicionários

Estrutura fundamental utilizada no agrupamento das filas finais de entrega. O dicionário foi implementado na função recursiva para mapear chaves correspondentes às categorias de prioridade (ex: "vip", "normal") para os seus respectivos deques, facilitando a separação das cargas.

2.4 Deque

Utilizado dentro do dicionário para representar a fila de cargas de cada categoria. As inserções são feitas com o método append(), o que garante complexidade O(1), tornando a deque mais eficiente que uma lista comum para esse caso de uso.

2.5 DataFrame (Pandas)

Utilizado para carregar o CSV, realizar agrupamentos e alimentar as visualizações gráficas de análise dos dados logísticos.

3. Recursão — Organização da Fila

3.1 Ordenação Multicritério (Merge Sort)

Em vez de uma ordenação simples, o sistema aplica o Merge Sort de forma sucessiva, aproveitando a estabilidade do algoritmo para criar uma hierarquia de prioridades de trás para frente:

- Custo (Desempate final): Ordenação decrescente pelo valor_frete.
- Prazo (Segundo critério): Ordenação crescente por prazo_dias.
- Criticidade (Critério principal): Ordenação decrescente por prioridade_cliente (VIP tem precedência sobre Normal).

3.2 Recursão

A função organizar_prioridade_recursivo distribui as cargas já ordenadas em suas respectivas filas baseadas nas categorias.

- Caso base: Lista de categorias vazia, retornando o dicionário pronto.
- Caso recursivo: A função pega a primeira categoria da lista, percorre as cargas adicionando as correspondentes ao deque atual, e faz a chamada recursiva passando o restante das categorias (categorias[1:]).

4. Análise de Complexidade (Big O)

Operação	Complexidade	Justificativa
Carregamento do CSV	O(n)	Percorre todas as n linhas uma vez
Conversão para tuplas	O(n)	Uma tupla criada por linha
Ordenação Multicritério	O(n log n)	O Merge Sort é executado três vezes. Como constantes são ignoradas no Big O (3 * n log n), a complexidade se mantém O(n log n).
Organização da fila (recursão)	O(n)	Percorre todas as n cargas para cada categoria constante
Inserção na deque	O(1)	Append é constante
Geração dos gráficos	O(n)	Percorre o DataFrame uma vez por gráfico

5. Análise Gráfica

5.1 Prazo Médio de Entrega por Destino

Gráfico de barras que exibe a média de dias (prazo_dias) necessários para entrega agrupados por cidade de destino.

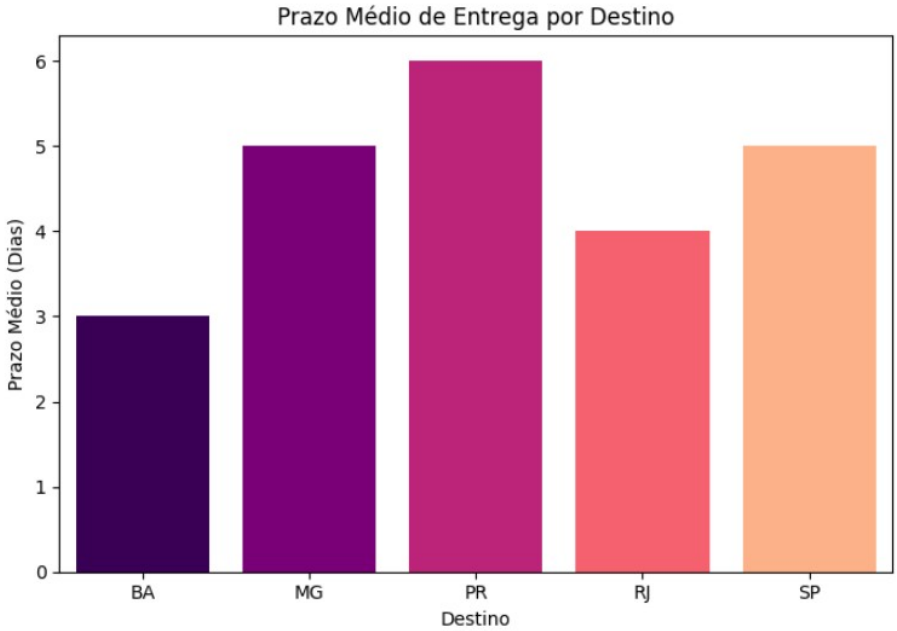

Figura 1 — Prazo Médio de Entrega por Destino

5.2 Relação Peso vs Valor do Frete

Gráfico de dispersão (Scatter plot) cruzando o peso_kg com o valor_frete, utilizando cores diferentes para separar clientes VIP e Normais. Permite analisar visualmente se cargas mais pesadas geram fretes proporcionalmente mais caros e o impacto da categoria do cliente nesse custo.

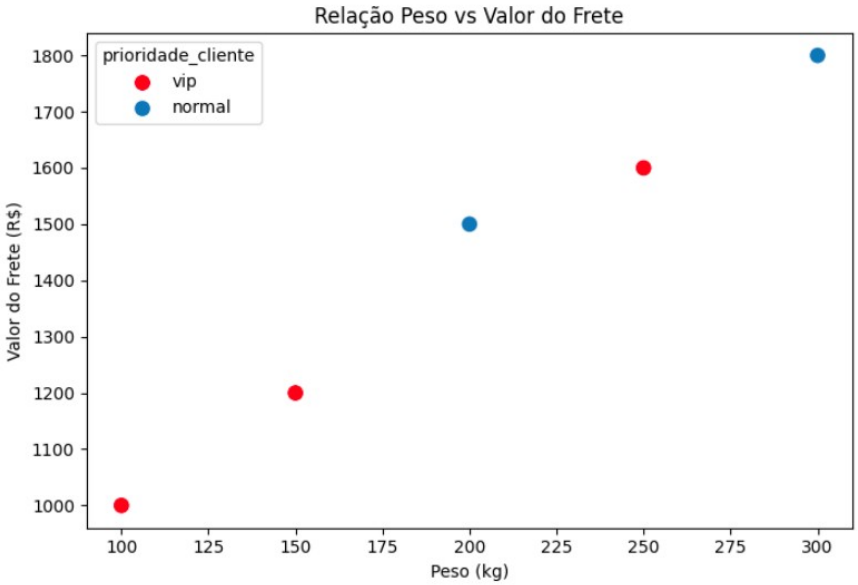

Figura 2 — Relação Peso vs Valor do Frete

6. Conclusão

A solução desenvolvida cumpre todos os requisitos do Checkpoint 1, aplicando de forma integrada as estruturas de dados exigidas: tuplas para imutabilidade das entregas, listas para armazenamento e fatiamento, dicionários no carregamento do CSV e agrupamentos das filas finais, deque para a fila de prioridade e DataFrame para análise e visualização.

A recursão implementada na função organizar_prioridade_recursivo garante a priorização correta das cargas por categoria, prazo e valor, com complexidade O(n) e inserções O(1) na deque. Os gráficos gerados com Seaborn e Matplotlib oferecem uma visão analítica clara dos dados logísticos trabalhados.